

Lecția 5

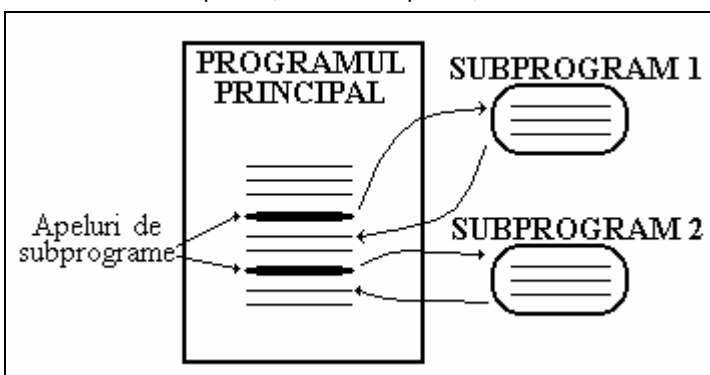
SUBPROGRAME

1. Funcții și proceduri

De multe ori e necesar ca o anumită secvență de instrucțiuni să se execute de mai multe ori în cadrul unui program, eventual pentru alte date. De aceea e bine ca aceste instrucțiuni să fie grupate într-un modul purtând un nume. El va fi numit **subprogram**.

Procedurile despre care vorbeam în primul capitol nu erau altceva decât niște subprograme. Astfel, ele sunt apelate din cadrul programului sau chiar din cadrul altui subprogram. Ambele se numesc **blocuri**. Singura condiție impusă este aceea ca *dacă un subprogram P apelează un subprogram Q, atunci se scrie întâi subprogramul Q și apoi subprogramul P*.

Când se apelează un subprogram, executarea continuă cu prima instrucțiune din respectivul subprogram. Când se termină executarea instrucțiunilor din subprogram, se continuă cu instrucțiunea imediat următoare apelului, din blocul apelant, conform schemei de mai jos.



Nu există restricții în apelul subprogramelor. Un subprogram poate fi apelat de mai multe ori în cadrul aceluiași bloc, chiar de el însuși, caz în care apelul este **recursiv**.

Există două tipuri de subprograme în limbajul Pascal: proceduri și funcții.

Funcțiile sunt subprograme care returnează o valoare, adică, la sfârșitul executării lor, în blocul apelant se dispune de o anumită valoare, valoarea returnată de respectiva funcție. Astfel, dacă avem o funcție *F*, un apel de funcție se poate folosi ca orice expresie, având tipul valorii returnate de funcție, deci poate fi folosit oriunde se poate folosi o expresie de acel tip: de exemplu: `x:=F; WriteLn(F)`. De fapt vom apela la funcțiile definite de noi așa cum apelăm, până acum, orice gen de funcție: *Sin*, *Trunc*, *Odd*, *Length*, *Pos* etc.. Procedurile sunt cazuri particulare de funcții, motiv pentru care prezentăm mai întâi funcțiile. Ele se comportă ca niște funcții oarecare, dar nu returnează valori. Aceasta nu înseamnă că nu pot comunica cu blocul apelant. Apelul procedurilor se face ca a oricăror proceduri folosite deja în programele noastre de până acum: `WriteLn`, `ReadLn`, `Insert` etc..

Folosind opțiunea de compilare `{SX+}` (*Extended syntax*), putem folosi funcțiile ca și proceduri, neinteresându-ne valorile returnate de ele, ci doar *ce fac* ele. Procedurile și funcțiile se pot apela între ele. Mai mult, putem avea atât proceduri, cât și funcții recursive.

Subprogramele se definesc, în întregime, în partea declarativă a unui program, deci înainte de corpul propriu-zis al programului, astfel:

```

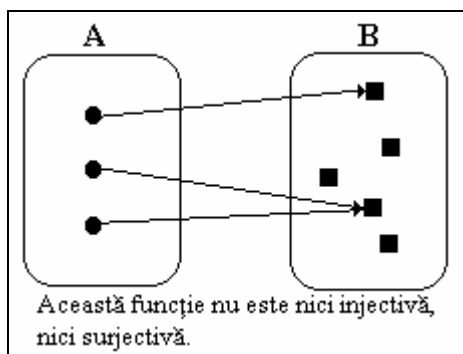
antet de subprogram;
declarații proprii subprogramului
begin
    instrucțiuni
end;

```

Antetul de subprogram diferă de la funcție la procedură.

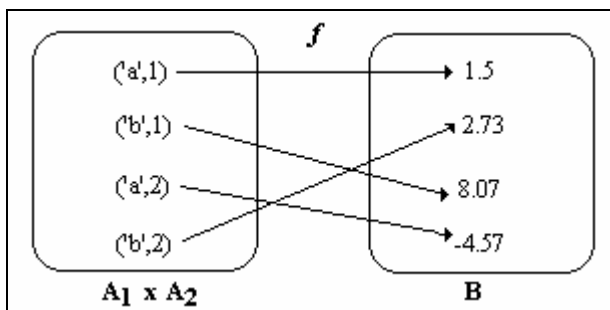
Funcții - o paralelă matematică - informatică

Vom reaminti noțiunea de funcție așa cum apare ea în matematică și vom prezenta diverse extinderi ale acesteia. Astfel, să considerăm două mulțimi *oarecare* (deci nu neapărat de numere !) A și B . În matematica elementară se numea *funcție* o lege f (o corespondență) prin care fiecărui element din A i se asociază un *unic* element din B . Astfel, această definiție nu exclude posibilitatea ca să existe în A două elemente cărora să le corespundă un același element din B . Dacă nu există un asemenea caz, se spune că funcția este *injectivă*. De asemenea, în B pot rămâne elemente care să nu corespundă nici unui element din mulțimea A . Dacă nu s-ar întâmpla așa ceva, funcția s-ar numi *surjectivă*. O funcție injectivă și surjectivă se numește funcție *bijectivă* sau *bijeție*.



A poate conține orice gen de elemente. Poate fi mulțimea numerelor întregi sau reale (*Integer*, *Real*), sau chiar o mulțime de caractere, șiruri de caractere sau valori logice (*Char*, *String*, *Boolean*). Astfel, am putea avea o funcție de genul $f: \text{Real} \rightarrow \text{Integer}$, ceea ce ar corespunde în matematică unei declarații $f: \mathbf{R} \rightarrow \mathbf{Z}$. Am putea avea și funcții de genul $f: \text{Char} \rightarrow \text{Boolean}$, fără corespondent în matematică. Această declarație ar corespunde unei funcții care are argumentul un caracter, iar rezultatul o valoare logică. (De pildă, am putea avea o funcție care să verifice dacă un caracter este sau nu o vocală.).

Datele din A ar putea fi chiar și tablouri sau înregistrări etc.. Nu același lucru este valabil și pentru mulțimea B , ale cărei elemente nu pot fi de tip structurat. În plus, domeniul funcției (A), ar putea reprezenta produsul cartezian a două mulțimi, A_1 , care să conțină elemente de un anumit tip t_1 , și A_2 , cu elemente de un alt tip t_2 . Astfel, un element din A ar fi o pereche ordonată de două elemente, primul din A_1 , iar al doilea din A_2 . De exemplu, fie $A_1 = \{ 'a', 'b' \}$, $A_2 = \{ 1, 2 \}$, iar $B = \mathbf{R}$. Avem: $f(('a', 1)) = 1.5$; $f(('b', 2)) = 2.73$; $f(('b', 1)) = 8.07$; $f(('a', 2)) = -4.57$.



Prezența celor două perechi de paranteze se datorează următoarelor:

- parantezele groase apar pentru a încadra argumentele funcțiilor;
- parantezele subțiri pentru a marca perechile ordonate ce sunt pe post de argumente.

În cele ce urmează vom renunța la una din perechile de paranteze, considerând că avem o funcție cu două argumente. E clar că putem să vorbim, în mod similar, de *funcții cu mai multe argumente, care pot fi chiar de tipuri diferite și nu neapărat numerice*.

În matematică a calcula $f(2)$ sau a calcula $f(x)$, unde x are valoarea 2 este același lucru. Se calculează $f(x)$, adică $f(2)$, iar x *nu-și modifică valoarea*, în urma apelului funcției. La fel se întâmplă și în informatică. Dar putem avea și cazul în care *argumentele funcției să-și modifice valorile lor inițiale*. Acest lucru nu ar putea fi posibil decât, firește, în cazul în care acele argumente ar fi doar *variabile*, nu și constante sau expresii mai complicate. În informatică determinarea valorii unei funcții poate fi însoțită de *efecte laterale*, lucru care nici nu se pune în discuție în matematică. Putem avea, de asemenea, funcții fără argument, dar care returnează valori și eventual, au unele efecte laterale. Valorile returnate pot depinde, în acest caz, de anumite teste asupra sistemului electronic de calcul (tastatură, memoria disponibilă, situații diferite ale discurilor).

Declararea funcțiilor. Parametri

O funcție numită `id_fun` se declară astfel:

```
function id_fun (v1,...vn: t1; w1,..., wm: t2; ...;
                 var z1,...,zp: t3; var u1, ..., uq: t4; ...): tip_f;
declarații locale;
begin
    instrucțiune 1;
    instrucțiune 2;
    .....
    instrucțiune k;
    id_fun := expresie
end;
```

În definiția de mai sus avem următoarele părți:

- antetul funcției:


```
function id_fun (v1,...vn: t1; w1, ..., wm: t2; ...;
                 var z1,...,zp: t3; var u1, ..., uq: t4; ...): tip_f;
```
- partea declarațiilor locale funcției: declarații locale;
- corpul funcției:


```
begin
    instrucțiune 1;
    instrucțiune 2;
    .....
    instrucțiune k;
    id_fun := expresie
end;
```

Antetul funcției cuprinde, pe lângă numele funcției (`id_fun`), două grupe de **parametri formali** (**argumente**). Prima conține mai multe liste de parametri ale căror valori se pot modifica în contextul funcției, dar rămân nemodificate în afara ei. Valorile parametrilor din listele precedate de cuvântul `var` se pot modifica, în sensul că după revenirea din apelul subprogramului, vom avea acces la noile valori (cele modificate în subprogram). **Parametrii efectivi** sunt acei parametri cu care se apelează funcția. Aceștia sunt puși în corespondență directă cu parametri formali ai funcției, în ordinea în care sunt scriși. Pentru listele din prima grupă, se pot transmite (ca parametri efectivi) orice gen de expresii de tipurile corespunzătoare, iar pentru parametri de după `var` nu se pot transmite decât variabile. Nu există restricții cu privire la alegerea identificatorilor parametrilor formali, așa încât putem folosi aceiași identificatori atât pentru parametri formali, cât și pentru cei efectivi (din blocul apelant). Prezența parametrilor nu este obligatorie.

Prin `tip_f` se înțelege **tipul rezultatului** returnat de funcție.

Partea declarațiilor locale funcției conține, ca și blocul programului principal, orice declarație de tipuri, constante sau variabile. Trebuie însă să nu existe riscul unor confuzii. Astfel, nu putem redeclara aici o variabilă ce apare și în lista parametrilor efectivi. Putem redeclara însă variabile care apar în blocul apelant cu același nume. În acest caz, în cadrul funcției, nu ne mai putem referi la variabilele respective din blocul apelant. De fapt există anumite reguli de vizibilitate a identificatorilor, asupra cărora vom reveni.

Corpul funcției cuprinde mai multe instrucțiuni. La funcții (nu și la proceduri), trebuie ca să existe (la sfârșit) o instrucțiune de atribuire: `id_fun:=expresie`, unde `expresie` este de tipul `tip_f`. Aceasta pentru ca funcția să returneze o valoare.

Exemple

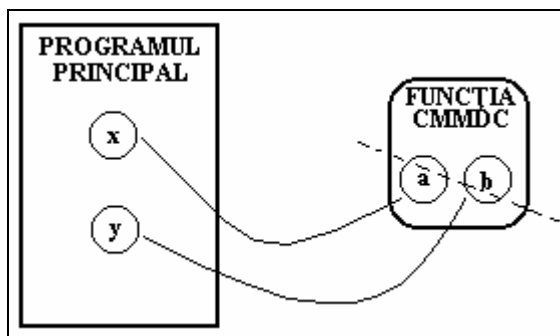
Pentru a clarifica noțiunile prezentate formal mai înainte, vom exemplifica.

1. Să scriem un program care să calculeze cel mai mare divizor comun a două numere. În acest scop, vom utiliza o funcție, care va avea două argumente: numerele întregi `a` și `b`. În urma algoritmului, `a` și `b` se modifică, însă acest lucru nu vrem să fie transmis înapoi în programul principal, așa încât nu vom folosi cuvântul rezervat `var` în fața identificatorilor. În partea stângă a textului programului am numerotat instrucțiunile, în ordinea în care se execută ele.

```

A      program CalculCMMDC;
      function CMMDC(a,b: Integer): Integer;
      begin
3          while a<>b do
              if a>b then a:=a-b
              else b:=b-a;
4          CMMDC:=a
      end;
      var x,y: Integer;
      begin
1          x:=20; y:=24;
2          WriteLn(CMMDC(x,y))
      end.

```



În blocul principal avem două variabile globale x și y , care se inițializează (în instrucțiunea 1) cu valorile întregi 20, respectiv 24. În instrucțiunea 2, avem un apel al procedurii `WriteLn`. Ea trebuie să afișeze o valoare întreagă, anume valoarea expresiei dintre paranteze. Aceasta este valoarea returnată de funcția `CMMDC`, care are parametri efectivi x și y , adică 20 și 24. De fapt, puteam scrie chiar și `CMMDC(20, y)` sau `CMMDC(x, 24)` sau `CMMDC(20, 24)`. Se intră, în acest moment, în funcția `CMMDC`, unde, în punctul A, se creează două noi variabile, cu numele a și b , care vor dispărea la sfârșitul funcției. Ele sunt parametrii formali ai funcției. a va primi valoarea lui x , deci 20, iar b pe cea a lui y , deci 24. După încheierea ciclului `while` (3), a și b vor avea ambele valoarea 4. Aceasta este valoarea care este returnată de funcție (în instrucțiunea de atribuire 4). Revenindu-se în blocul principal al programul, se va afișa, numărul 4. Variabilele x și y nu sunt afectate.

2. Următorul exemplu va face precizări asupra variabilelor care apar în blocul principal, parametrii funcției și variabilele locale ei. Se calculează produsul $n! = 1 \times 2 \times \dots \times n$.

```

program Test1;
function Factorial(n: Integer): Integer;
var i, p: Integer;
begin
    p:=1; for i:=1 to n do p:=p*i;
B      Factorial := p
end;
var p, i: Integer;
begin
    p:=3;
C      i:=Factorial(p);
      WriteLn(i)
end.
  
```

Programul va afișa 6 ($=3!$). Ce se întâmplă? În primul rând, avem declarată o funcție `Factorial`, având parametrul formal n . În interiorul funcției, se calculează produsul $p = n!$. Pentru aceasta se folosește încă variabilă locală i . La sfârșit, funcția returnează valoarea lui p , iar i , p și n dispar.

Execuția programului se desfășoară în felul următor: variabila globală p primește valoarea 3. Se apelează funcția de calcul a factorialului, cu p global drept parametru efectiv. Astfel, parametrul formal n va primi valoarea 3. Acest p global nu are nimic comun cu variabila locală funcției, purtând același nume. Deci, când execuția ajunge în punctul B, p -ul global va avea valoarea 3, iar cel local valoarea 6. În funcție apare o altă variabilă i , dar acest i nu este tot una cu variabila globală i , care primește valoarea factorialului lui 3, deci 6.

3. Să scriem, în continuare un program care să calculeze recursiv factorialul, adică ținând cont de formulele: $n!=1$, dacă $n=0$ și $n! = (n-1)! \times n$, dacă $n>0$. Considerăm:

```

function Fact(n: Integer): Integer;
begin
    if n=0 then Fact:=1 else Fact:=Fact(n-1)*n
end;
  
```

În atribuirea $\text{Fact} := \text{Fact}(n-1) * n$ am folosit identificatorul Fact în două feluri diferite: $\text{Fact} := \dots$ înseamnă că se pregătește valoarea ce trebuie returnată de funcția Fact , iar $\text{Fact}(n-1)$ înseamnă un apel (recursiv) al acestei funcții pentru parametrul efectiv $n-1$. Astfel, a calcula $\text{Fact}(2)$ înseamnă: Este $2=0$? Nu, atunci $\text{Fact}(2)$ va fi $\text{Fact}(1)$ înmulțit cu 2. Dar $\text{Fact}(1)$ se determină exact la fel. Este $1=0$? Nu, atunci $\text{Fact}(1)$ va fi $\text{Fact}(0)$ înmulțit cu 1. Dar $\text{Fact}(0)$ se determină exact la fel. În sfârșit, n (adică 0) este egal cu zero, așa încât obținem $\text{Fact}(0)=1$. În acest moment, avem o reîntoarcere la $\text{Fact}(1)$, obținând $\text{Fact}(1)=1$ și transmițând această valoare lui $\text{Fact}(2)$, care o va înmulți cu 2 și va obține rezultatul 2. Deci: $\text{Fact}(2) = \text{Fact}(1) \times 2 = (\text{Fact}(0) \times 1) \times 2 = (1 \times 1) \times 2 = 2$.

Propunem cititorului redactarea întregului program.

4. În cele ce urmează vom prezenta un exemplu ceva mai complex. Vom scrie o funcție pentru determinarea celui mai mare divizor comun al n numere întregi. Funcția va fi recursivă, conform relației: $\text{CMMDC}(x_1, \dots, x_{n-1}, x_n) = \text{CMMDC}(\text{CMMDC}(x_1, \dots, x_{n-1}), x_n)$.

Astfel, vom scrie două funcții. Prima va determina cel mai mare divizor comun a două numere întregi a și b . Cu cea de a doua, recursivă, se va determina cel mai mare divizor comun al n numere întregi, astfel: se va determina d =cel mai mare divizor comun al primelor $n-1$ numere, apoi se va determina cel mai mare divizor comun al numerelor d și x_n . Vom memora numerele într-un vector.

```

program Calcul_CMMDC_n_numere;
type vector = array[1..10] of Integer;
var a: vector; x, n, i: Integer;
function CMMDC2(a,b: Integer): Integer;
begin
    while a<>b do if a>b then a:=a-b else b:=b-a;
    CMMDC2:=a
end;
function CMMDC(x: vector; n: Integer): Integer;
begin
    if n=2 then CMMDC:=CMMDC2(x[1],x[2])
    else CMMDC:=CMMDC2(CMMDC(x,n-1),x[n])
end;
begin
    Write('Dati numarul de elemente: '); ReadLn(n);
    for i:=1 to n do
        begin Write(' - dati numarul al ',i,'-lea: '); ReadLn(a[i]) end;
    x:=CMMDC(a,n);
    WriteLn('Cel mai mare divizor comun este = ',x);
    ReadLn
end.

```

În programul precedent ne-am jucat atât cu identificatorii de variabile, cât și cu apelurile dintr-o funcție a unei alte funcții și, de asemenea, cu recursivitatea. Observăm că, deoarece funcția CMMDC apelează funcția CMMDC2 , aceasta din urmă a fost scrisă înaintea primeia. În blocul principal, a este un vector, iar x un număr întreg. În cadrul funcției CMMDC2 , a este un număr întreg, iar în cadrul funcției CMMDC x este un vector. Pentru datele de intrare $n=3$, $a=(10,14,6)$ se va afișa 2.

5. Funcția CMMDC2 de mai sus se poate scrie și în alte două moduri, ambele bazându-se pe **algoritmul clasic al lui Euclid**. Cea de a doua variantă, recursivă, se bazează pe relația matematică: $\text{cmmdc}(a,b) = \text{cmmdc}(b, a \bmod b)$.

a) Varianta iterativă:

```

function CMMDC2(a,b: Integer): Integer;
var aux: Integer;
begin
    while b<>0 do begin aux:=b; b:=a mod b; a:=t end;
    CMMDC2:=a
end;

```

b) *Varianta recursivă:*

```
function CMMDC2(a,b: Integer): Integer;
begin
    if b=0 then CMMDC2:=a else CMMDC2:=CMMDC2(b, a mod b)
end;
```

6. În continuare, punem în evidență situațiile în care modificările argumentelor se transmit în exteriorul funcției. În această situație, nu putem apela funcțiile decât cu variabile. Să scriem, în două variante, o funcție care să realizeze următorul lucru: să pună în fața și în spatele unui șir de caractere câte un semn '#', apoi să returneze lungimea șirului nou format. De pildă, pentru șirul 'abc' să se obțină valoarea 5, efectul lateral al funcției fiind transformarea șirului în '#abc#'. Vom observa că în prima din variante nu obținem rezultatul așteptat, tocmai fiindcă parametrii formali ai funcției nu au fost declarați ca fiind variabili. Însă a doua variantă funcționează corect.

```
1) function Gresit(S: String): Integer;
    begin
        S:='#'+S+'#'; Gresit := Length(S)
    end;
2) function Corect(var S: String): Integer;
    begin
        S:='#'+S+'#'; Corect := Length(S)
    end;
```

Să numim *F* oricare din funcțiile anterioare. Un program de test ar putea fi:

```
program Test2;
function F(...):...;
begin
    .....
end;
var S: String;
begin
    S:='abc';
    WriteLn(F(S));
    ReadLn
end.
```

În prima situație putem apela direct funcția *Gresit* prin *Gresit('abc')*. Dar un apel de genul *Corect('abc')* va semnaliza eroare sintactică!

Când efectuăm un apel în care parametrii formali nu sunt precedați de cuvântul *var*, se spune că are loc un **apel prin valoare**. În celălalt caz, avem un **apel prin referință**. Într-adevăr, în al doilea caz, parametrii formali nu se creează în altă parte, în cadrul funcției, ci *se suprapun* peste cei efectivi.

Deci, pe când în cazul apelului prin valoare comunicarea este doar în sensul de la blocul apelant la cel apelat, în cazul apelului prin referință comunicarea se realizează în ambele sensuri.

Un exemplu reprezentativ pentru apelul prin referință este cazul interschimbării a două variabile. Vom scrie o astfel de funcție a carei valoare returnată nu contează.

```
function Schimba(var a,b: Integer): Boolean;
var aux: Integer;
begin
    aux:=a;
    a:=b;
    b:=aux
end;
```

Un programul complet folosind această funcție este:

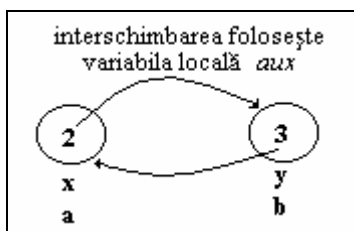
```
{ $X+ }
program Test3;
function Schimba(...):...
    .....
```

```

.....
var x,y: Integer; b: Boolean;
begin
  x:=2; y:=3;
  Schimba(x,y); { am putea scrie si b := Schimba(x,y) }
  WriteLn(x:3, y:3);
end.

```

Programul va afișa 3, apoi 2. Ce se întâmplă, de fapt? Variabilele globale *x* și *y* primesc valorile 2, respectiv 3. În momentul în care se apelează funcția *Schimba* (care se apelează ca o procedură, datorită opțiunii de compilare de la începutul programului!), nu se creează două noi variabile *a* și *b*, ci *x* și *y* primesc încă un nume. Astfel, orice acțiune asupra lui *a* și *b* se va repercuta și asupra lui *x* și *y*.



În concluzie, ca o regulă generală, atunci când se dorește ca o procedură să modifice valoarea unui argument, acesta se va declara ca *parametru valoare*, altfel ca *parametru variabilă (referință)*. Totuși, când folosim structuri de date de dimensiuni considerabile, utilizarea parametrului valoare ocupă atât spațiu de memorie, cât și timp de execuție. Este preferabil, în această situație, declararea parametrului respectiv ca variabil; printr-un comentariu se poate preciza ca procedura să nu îi modifice valoarea.

7. În continuare, vom scrie un program care să afișeze toate numerele prime mai mici sau egale cu un număr *n* dat. Vom scrie, o funcție *EstePrim* care verifică primalitatea unui număr întreg *x*. Această funcție va apela la o funcție proprie de calcul a radicalului unui număr real *x*, pe care o vom numi sugestiv *Radical*. Dacă vrem să dispunem de funcția *Radical* și în blocul principal, atunci o vom scrie în cadrul său. Singura restricție va fi aceea ca funcția *Radical* să apară înainte de funcția *EstePrim*.

Nu vom proceda, însă, așa, ci vom declara și defini funcția *Radical* în cadrul (restrâns) al funcției *EstePrim*. Astfel, nu vom putea dispune în altă parte de această funcție! Acest mod de realizare a programelor se numește *modularizat* și vom reveni asupra subiectului.

```

program NumerePrime;
function EstePrim(x: Integer): Boolean;
  function Radical(xx: Real): Real;
  const eps=0.0001;
  var an,an1: Real;
  begin
    an1:=1; an:=(1+xx)/2;
    while abs(an-an1)>eps do begin an1:=an; an:=(an+xx/an)/2 end;
    Radical:=an
  end; {Radical }
  var d,rad: Integer; prim: Boolean;
  begin {functia EstePrim}
    if x mod 2 = 0 then if x=2 then prim:=True else prim:=False
    else
      begin
        d:=3; prim:=True; rad:=Trunc(Radical(x));
        while (d<=rad) and prim do
          if x mod d = 0 then prim:=False else d:=d+2
        end;
      end;
    end;
  end;
end;

```



```

EstePrim:=prim
end; {EstePrim }
var n,i: Integer;
begin { Program principal }
  WriteLn('Determinare numere prime'); Write('Dati n = '); ReadLn(n);
  for i:=2 to n do if EstePrim(i) then Write(i:3,' ', ' '); ReadLn
end.

```

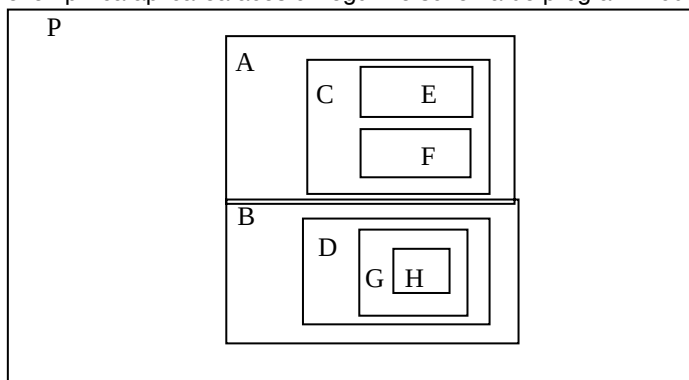
Modularizarea. Domenii de vizibilitate a identificatorilor

După cum am spus, un program în Pascal este un **bloc** (modul). Un bloc, la rândul său, poate conține alte blocuri (module) ș.a.m.d., adică ele pot fi imbricate unele în altele. Fiecare bloc va fi desemnat prin identificatorul funcției, procedurii sau al programului corespunzător.

Există trei reguli de valabilitate sau vizibilitate a identificatorilor:

- În cadrul unui bloc, un identificador poate fi declarat o singură dată.
- Un același identificador poate fi declarat în blocuri diferite. În acest caz se aplică regula:
- Natura unui identificador *id* care apare într-o instrucțiune se stabilește căutând cel mai interior bloc ce include atât instrucțiunea, cât și declararea lui *id*.
- Nu se poate folosi un identificador în exteriorul blocului în care a fost declarat.

Pentru a exemplifica aplicarea acestor reguli fie schema de program modularizat:



Putem vorbi de o ierarhizare a blocurilor pe nivele:

Bloc	Nivel
P	0
A,B	1
C,D	2
E,F,G	3
H	4

Domeniile de vizibilitate (valabilitate) ale identificatorilor sunt:

Identificatorii declarați în blocul:	sunt accesibili în blocul:
P	P, A, B, C, D, E, F, G, H
A	A, C, E, F
B	B, D, G, H
C	C, E, F
D	D, G, H
E	E
F	F
G	G, H
H	H

De exemplu, o variabilă declarată în blocurile B și G și apelată în blocul H va avea tipul precizat în declarația conținută de blocul G.

Proceduri

Conceptul de procedură este familiar cititorului care a parcurs cu atenție lecțiile primului capitol. După cum am spus deja, **procedura** este o funcție particulară, ea nu returnează valori. Astfel, identificatorul ei nu poate reprezenta decât procedura însăși, (nu și vreo valoare returnată). O procedură va fi apelată prin identificatorul ei urmat de lista parametrilor efectivi. Declararea și definirea unei proceduri se face astfel:

```
procedure id_proc (lista_de_parametri_formali);
  declarații locale;
begin
    instrucțiuni
end;
```

Apelul se va realiza prin `id_proc(lista de param efectivi)`. Pot exista atât parametri variabili (referință), cât și parametri valoare. Pot exista proceduri fără parametri. La o procedură contează efectele laterale și valorile returnate, prin intermediul parametrilor referință.

În lecțiile anterioare am utilizat proceduri predefinite. Astfel, de exemplu `WriteLn` are parametrii formali transmiși prin valoare. Procedura `ReadLn` are parametrii transmiși prin referință, tocmai pentru că ei trebuie să se modifice (actualizându-se la valorile citite de la tastatură).

Ideii de *programare structurată* i se adaugă acum cea de **programare modularizată**, care permite realizarea unor programe foarte flexibile și eficiente, bine structurate. Mai târziu vom vorbi despre **programarea orientată pe obiecte**, care face ca și datele programelor să fie bine structurate.

2. Aplicații ale subprogramelor

1. Admiterea în liceu.

Pentru fiecare din etapele importante ale programului, vom defini câte o procedură. Procedurile vor fi apelate, pe rând, conform selecției instrucțiunii `case` din blocul principal. Apelurile se vor realiza într-un ciclu. Acesta se va termina când se va opta pentru opțiunea 's'.

```
program AdmitereLiceu;
{ varianta cu "record" si proceduri}
uses Crt;
type elev = record nume: String; nota1,nota2,media: Real end;
type vector=array[1..20] of elev;
var elevii: vector; NrElevi: Integer; optiune: Integer; date_introduse:
Boolean;
procedure IntroducereDateElevi(var E:vector; var n: Integer);
var i: Integer;
begin
  if not date_introduse then optiune:=1
  else begin
    WriteLn('Atentie ! Aveti datele deja introduse !');
    WriteLn('Doriti reintroducerea lor ? (1=da, 2=nu)');
    ReadLn(optiune);
  end;
  if optiune=1 then
  begin
    Write('Dati numarul de elevi : '); ReadLn(n);
    for i:=1 to n do
      with E[i] do
        begin
          WriteLn('Dati numele si prenumele !'); ReadLn(nume);
          WriteLn('Dati notele !'); ReadLn(nota1, nota2);
```

```

        media:=(notal+nota2)/2
    end;
    date_introduse:=True
end
end;
procedure ListareElevi(E: vector; n: Integer);
var i,j: Integer; numele: String;
begin
    for i:=1 to n do
        with E[i] do
            begin { se completeaza numele sau cu spatii la dreapta }
                numele:=nume;
                for j:=1 to 40-Length(nume) do numele:=numele+' ';
                WriteLn(i:2,'. ',numele,' : ', notal:6:2,
                    nota2:6:2, media:6:2)
            end
        end
    end;
procedure OrdonareDupaMedii(n: Integer; var E: vector);
var i,j: Integer; aux: elev;
begin
    if not date_introduse then
        WriteLn('Introduceti, mai intai datele elevilor!')
    else
        begin
            for i:=1 to Pred(n) do
                for j:=Succ(i) to n do { comparatia se face dupa medii }
                    if E[i].media < E[j].media then
                        { are loc interschimbarea elevilor }
                        begin aux:=E[i]; E[i]:=E[j]; E[j]:=aux end;
                        ListareElevi(E,n)
                    end
                end
            end;
procedure OrdonareAlfabetica(n: Integer; var E: vector);
var i,j: Integer; aux: elev;
begin
    if not date_introduse then
        WriteLn('Introduceti, mai intai datele despre elevi !')
    else
        begin
            for i:=1 to Pred(n) do
                for j:=Succ(i) to n do { comparatia se face dupa nume }
                    if E[i].nume > E[j].nume then
                        { are loc interschimbarea elevilor }
                        begin aux:=E[i]; E[i]:=E[j]; E[j]:=aux end;
                        ListareElevi(E,n)
                    end
                end
            end;
procedure Meniu;
begin
    ClrScr; WriteLn('ORDONARE ELEVII LA CONCURS'); WriteLn;
    WriteLn('0 = oprire program');
    WriteLn('1 = introducere date elevi');
    WriteLn('2 = listare dupa medii');
    WriteLn('3 = listare alfabetica');
    WriteLn; Write('Optiunea dvs. = '); ReadLn(optiune);
end;
begin
    date_introduse:=False;
    repeat
        Meniu;
        case optiune of
            1: IntroducereDateElevi(elevii, NrElevi);
            2: OrdonareDupaMedii(NrElevi, elevii);
            3: OrdonareAlfabetica(NrElevi, elevii)
        end;
    end;
end;

```

```

    WriteLn; WriteLn('Apasati <Enter> pentru continuare !'); ReadLn
until optiune=0
end.

```

2. Simplificarea unei fracții. Să se simplifice o fracție a/b .

Vom proceda în felul următor: dându-se două numere a și b , vom verifica dacă b este diferit de zero sau nu. În caz că nu, se va afișa un mesaj de eroare. Altfel, dacă a nu este nul, se va determina, cu o funcție corespunzătoare, cel mai mare divizor comun a lui a și b . Fie acesta c . Rezultatul va fi dat sub forma u/v , unde u și v se determină împărțind pe a și b la c . Dacă $v=1$, atunci vom afișa doar pe u .

```

program SimplificareaUneiFractii;
function cmmdc(a,b: Integer): Integer;
begin
    while a<>b do if a>b then a:=a-b else b:=b-a;
    cmmdc:=a
end;
procedure Simplifica(a,b: Integer; var u,v: Integer);
var c: Integer;
begin
    c:=cmmdc(a,b); u:=a div c; v:=b div c
end;
var a,b,u,v: Integer;
begin
    WriteLn('Simplificare'); WriteLn('Dati a si b !');
    ReadLn(a,b);
    if b=0 then WriteLn('Fractie inexistentă !')
    else if a=0 then WriteLn('Rezultat: 0')
    else begin
        Simplifica(a,b,u,v);
        if v=1 then WriteLn('Rezultat: ',u)
        else WriteLn('Rezultat: ',u,'/',v)
    end;
    ReadLn
end.

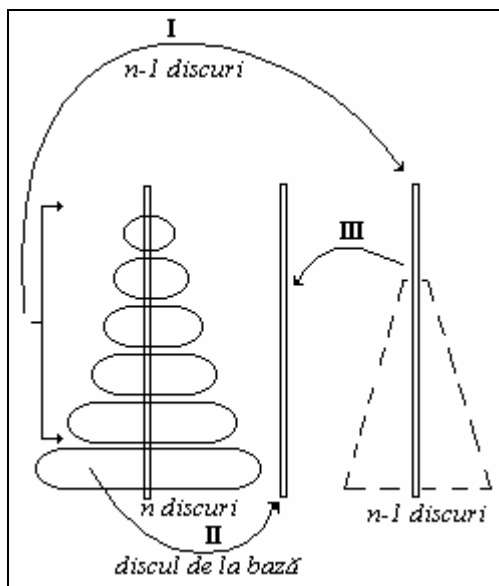
```

3. Turnurile din Hanoi. Se spune că în Vietnamul antic, în Hanoi, erau trei turnuri, pe unul din ele fiind puse, în ordinea descrescătoare a diametrelor lor, mai multe (opt) discuri de aur. Din motive obiective, niște călugări, care le aveau în grijă, trebuiau să așeze discurile pe cel de-al doilea turn, în aceeași ordine. Ei puteau să folosească, eventual, turnul al treilea, deoarece, altfel discurile nu ar fi avut stabilitate. Discurile puteau fi mutate unul câte unul. De asemenea, se renunța din start la ideea de a așeza un disc mai mare peste unul mai mic, deoarece exista riscul ca să se strice discurile, din cauza diferenței mari de masă.

Deși, aparent simplă, după câteva încercări, cu un prototip în față, cititorul va constata că problema nu e banală. Însă este posibilă o rezolvare optimă a ei (cu numai $2^n - 1$ mutări, deci 255, pentru $n=8$), folosind tehnica recursivă "*divide et impera*", asupra căreia vom reveni în capitolul următor.

Problema este de a muta n discuri de la turnul 1 la turnul 2. Pentru a o rezolva, să vedem cum se mută, în general, m discuri de la un turn p la un turn q . Se mută primele $m-1$ discuri de pe p pe r , r fiind turnul auxiliar, apoi singurul disc rămas pe p (discul cel mai mare) se mută de pe p pe q , după care cele $m-1$ discuri sunt mutate de pe r pe q . Firește, mutarea celor $m-1$ discuri de la p la r și de la r la q se realizează la fel, deci recursiv. Mutarea primelor $m-1$ discuri este corectă, deoarece existența discurilor de diametre mai mari, la bazele celor trei turnuri nu afectează cu nimic mutările discurilor mai mici.

În cazul limită $m=1$, avem doar o mutare a discului din vârful turnului p spre q .



Programul de mai jos soluționează (cu animație) problema descrisă, pentru $n=8$. Procedura de bază este `Han`, iar celelalte proceduri sunt pentru mișcarea discului curent. Există și două proceduri cu structură inedită. Ele sunt scrise în limbaj de asamblare. Folosind întreruperea `10h`, realizează ascunderea, respectiv reafisharea cursorului text.

```

program TurnurileDinHanoi;
uses Crt;
const Pauza=10; forma= #219; Virf: array [1..3 ] of Byte=(13,22,22);
procedure HideCursor; assembler; { ascunde cursorul pilpăitor, in modul
text }
asm MOV AX,$0100; MOV CX,$2607; INT $10 end;
procedure ShowCursor; assembler; { reafiseaza cursorul }
asm MOV AX,$0100; MOV CX,$0506; INT $10 end;
function ColTija (tija : Byte) : Byte; {stabileste coloana unei tije}
begin ColTija := 24*tija-8 end;
procedure MutaDreapta (disc, tija1, tija2 : Byte);
var i,k: Byte;
begin
  for i := ColTija(tija1)-disc to Pred(ColTija(tija2)-disc) do
    begin
      Delay(Pauza); if KeyPressed then Halt(1);
      GoToXY(i,3); for k:=0 to 2*disc do Write(' ');
      GoToXY(i+1,3); for k:=0 to 2*disc do Write(forma)
    end
  end;
procedure MutaStanga (disc, tija1, tija2 : Byte);
var i,k: Byte;
begin
  for i := ColTija(tija1)-disc downto Succ(ColTija(tija2)-disc) do
    begin
      Delay(Pauza); if KeyPressed then Halt(1);
      GoToXY(i,3);
      for k:=0 to 2*disc do Write(' ');
      GoToXY(i-1,3);
      for k:=0 to 2*disc do Write(forma)
    end
  end;
procedure Coboara (disc, tija : Byte);
var i,k: Byte;

```

```

begin
  for i := 3 to Pred(Virf[tija]-1) do
    begin
      Delay(Pauza); if KeyPressed then Halt(1);
      GoToXY(ColTija(tija)-disc,i);
      for k:=0 to 2*disc do Write(' ');
      GoToXY(ColTija(tija)-disc,i+1);
      for k:=0 to 2*disc do Write(forma)
    end;
    Dec(Virf[tija])
  end;
procedure Ridica (disc, tija : Byte);
var i,k: Byte;
begin
  for i := Virf[tija] downto 4 do
    begin
      Delay(Pauza); if KeyPressed then Halt(1);
      GoToXY(ColTija(tija)-disc,i);
      for k:=0 to 2*disc do Write(' ');
      GoToXY(ColTija(tija)-disc,i-1);
      for k:=0 to 2*disc do Write(forma)
    end;
    Inc(virf[tija])
  end;
procedure Muta(disc, tija1, tija2 : Byte);
begin
  Ridica(disc,tija1);
  if (tija1 < tija2) then
    MutaDreapta(disc,tija1,tija2)
  else
    MutaStanga(disc,tija1,tija2);
  Coboara(disc,tija2)
end;
procedure Han(n, tija1, tija2, tija3 : Byte);
begin
  if (n = 1) then Muta(1,tija1,tija2)
  else
    begin
      Han(n-1,tija1,tija3,tija2);
      Muta(n,tija1,tija2);
      Han(n-1,tija3,tija2,tija1)
    end
end;
procedure Initializari;
var k,disc: Byte;
begin
  HideCursor; ClrScr;
  for disc:=1 to 9 do
    begin
      GoToXY(ColTija(1)-disc,Virf[1]+disc-1);
      for k:=0 to 2*disc do
        Write(forma);
      end
    end;
end;
begin { PROGRAM }
  Initializari; GoToXY(28,1); WriteLn(' ~Turnurile din Hanoi~ ');
  Han(8,1,2,3); ShowCursor
end.

```

Citorii care nu cunosc elemente de limbaj de asamblare vor considera procedurile ShowCursor și HideCursor (și altele asemănătoare) ca atare. Mai puțin importantă este înțelegerea modului cum sunt ele realizate și mai mult ce execută ele.

În cadrul unui text sursă Pascal, pot apare secvențe de limbaj de asamblare, încadrate de cuvintele **asm** și **end**. Dacă o procedură este scrisă în întregime în limbaj de asamblare, se va preciza aceasta prin cuvântul rezervat **assembler**. Așa am procedat și noi.

Procedura **Halt** (apărută în program) va determina oprirea forțată a programului.

3. Variabile inițializate. Variabile de tip funcție sau procedură

Variabile inițializate

Un element surpriză în programul anterior (din secțiunea 2) îl constituie declarația:

```
const Virf: array [1..3] of Byte=(13,22,22);
```

Este vorba despre definirea unei variabile (nu a unei constante), care se inițializează la o anumită valoare. Astfel, când vrem să inițializăm vectori se vor scrie elementele acestuia într-o listă încadrată de două paranteze. Când avem matrice, avem o listă de liste, cuprinse între paranteze, iar în cazul variabilelor simple vom avea valorile, pur și simplu. Un caz special îl constituie înregistrările, asupra cărora vom reveni atunci când ne vom reîntâlni cu această problemă.

Propunem cititorului să îmbunătățească aplicația 2 din secțiunea 2, astfel încât meniul să fie mai atractiv. Comenzile meniului să aibe asociate niște explicații. Comenzile și explicațiile asociate să fie memorate direct în variabile vectori inițializați.

Variabile de tip subprogram

Pentru a determina minimul, respectiv maximul, dintr-un vector de elemente oarecare se procedează similar, singura deosebire fiind de un semn ("<" sau ">" după caz). Astfel, să unificăm cele două relații de inegalitate sub numele unei funcții generice *Relatie*, care având două argumente numere întregi *x* și *y* să întoarcă *True* sau *False*, după cum între *x* și *y* este stabilită sau nu relația dorită.

Limbajul Pascal ne permite să folosim pe *Relatie* ca pe o variabilă de tip funcție și chiar ca parametru în apelul unui subprogram. Determinarea minimului și maximului dintr-un vector se va face conform programului de mai jos, în care trebuie să observăm prezența opțiunii de compilare `{ $F+ }`, care ne permite să folosim variabile de tip subprogram.

```
{ $F+ }
program VariabileSubprogram;
uses Crt;
type functie_de_comparatie = function(x,y: Integer):Boolean;
vector = array[1..10] of Integer;
var x: vector; i,n, optim: Integer; Relatie: functie_de_comparatie;
function MaiMic(x,y: Integer): Boolean;
begin MaiMic := x<y end;
function MaiMare(x,y: Integer): Boolean;
begin MaiMare := x>y end;
begin
  WriteLn('Determinare minim sau maxim');
  Write('n = '); ReadLn(n);
  for i:=1 to n do begin Write('x[' ,i ,']= '); ReadLn(x[i]) end;
  WriteLn('Ce optim doriți sa determinam ?');
  WriteLn('m = minim, M = maxim');
  if ReadKey='m' then Relatie:=MaiMic else Relatie:=MaiMare;
  optim:=x[1];
  for i:=2 to n do if Relatie(x[i],optim) then optim:=x[i];
  WriteLn('Optimul determinat este = ',optim); ReadLn
end.
```

Asupra acestui tip special de variabile vom reveni în lecția 11, unde vom prezenta o procedură de reprezentare grafică a unei funcții oarecare, al cărei corp este prezent în textul programului.

4. Recursivitate

Noțiunea de recursivitate (sau recursie) vă este deja familiară, am folosit-o până acum de mai multe ori.

În continuare ne vom ocupa de:

- condiția de consistență a unei definiții recursive;
- utilizarea stivei în recursivitate;
- tipurile de recursivitate: directă și indirectă.

Condiția de consistență a unei definiții recursive

Să considerăm definiția recursivă de mai jos (*funcția lui Ackermann*):

$$A(m,n) = \begin{cases} n+1, & \text{daca } m=0, \\ A(m-1,1), & \text{daca } n=0, \\ A(m-1, A(m,n-1)), & \text{altfel.} \end{cases}$$

În limbajul Pascal, această funcție se va scrie:

```
function A(m,n: Integer);
begin
    if m=0 then A:=n+1
    else if n=0 then A:=A(m-1,1) else A:=A(m-1,A(m,n-1))
end;
```

Să determinăm $A(2,2)$. Vom aplica succesiv definiția până la identificarea unor argumente pentru care valoarea funcției A se poate calcula. Apoi vom substitui valoarea calculată în locul celui mai interior argument de pe poziția a doua, după care vom relua aplicarea definiției, dacă numele funcției nu mai apare. Avem succesiv:

```
A(2,2)=A(1,A(2,1))=A(1,A(1,A(2,0)))=A(1,A(1,A(1,1)))=A(1,A(1,A(0,A(1,0))))=A(1,A(1,A(0,A(0,1))))=A(1,A(1,A(0,A(0,2))))=A(1,A(1,A(0,A(0,A(1,0)))))=A(1,A(0,A(0,A(0,A(0,1)))))=A(1,A(0,A(0,A(0,A(0,2)))))=A(1,A(0,A(0,A(0,3))))=A(1,A(0,A(0,4)))=A(1,5)=A(0,A(1,4))=A(0,A(0,A(1,3)))=A(0,A(0,A(0,A(1,2))))=A(0,A(0,A(0,A(0,A(1,1)))))=A(0,A(0,A(0,A(0,A(0,A(1,0)))))=A(0,A(0,A(0,A(0,A(0,1)))))=A(0,A(0,A(0,A(0,A(0,A(0,2)))))=A(0,A(0,A(0,A(0,A(0,3)))))=A(0,A(0,A(0,A(0,4))))=A(0,A(0,5))=A(0,6)=7.
```

Observăm tendința de identificare a unor valori direct calculabile, urmată de utilizarea lor în vederea obținerii valorilor căutate.

Astfel, o definiție recursivă trebuie să satisfacă următoarea condiție: *valoarea funcției trebuie să fie ori direct calculabilă, ori calculabilă cu ajutorul unei valori direct calculabile*. Această condiție se numește **condiția de consistență**, iar ea este verificată atât de funcția lui Ackermann, cât și de funcția recursivă cu ajutorul căreia calculasem factorialul unui număr.

Un exemplu de funcție inconsistentă este următoarea:

```
function Inconsistent(n: Integer);
begin
    if n=0 then Inconsistent:=1
    else Inconsistent:=n*Inconsistent(n+1)
end;
```

Utilizarea stivelor în recursivitate

Mai întâi trebuie să precizăm că, în informatică, prin stivă se înțelege ceea ce se înțelege și în viața de zi cu zi. Deocamdată vom considera o stivă ca fiind un vector asupra căruia se pot face operațiile: adăugarea unui element, după ultimul element (indicat de un număr n), și eliminarea

ultimului element (al n -lea) din vector. Vectorul ar putea fi asemuit unei stive de cărți: putem pune o carte nouă peste celelalte deja existente și putem să luăm cartea de deasupra. Atunci când, de exemplu, se dorește calcularea valorii $3!$, se memorează într-o stivă, inițial vidă, acest număr 3, necesar pentru a fi înmulțit cu $2!$. Procesul continuă până la $0!$, când are loc eliminarea, pe rând, a elementelor din stivă, simultan cu câte o înmulțire.

```
function Fact(n: Integer);
begin
    if n=0 then Fact:=1 else Fact:=Fact(n-1)*n
end;
```

Mediul Turbo-Pascal dispune de o stivă proprie, cu o anumită dimensiune. Depășirea acesteia se face cu opțiunea de compilare {\$S+}. Ea este implicită. Când lucrăm cu apeluri recursive ce încarcă foarte mult stiva, dar avem condiția de consistență a recursiei îndeplinită, este bine să folosim opțiunea de compilare {\$S-}.

Noi înșine putem simula recursivitatea, implementând o structură proprie de stivă. Următorul program calculează factorialul unui număr, folosind o iterație asupra unei stive, iterație ce simulează recursivitatea.

```
program SimulareRecursivitate;
const max=20;
type stiva=record
    x: array[1..max] of Integer; n: Integer
end;
procedure InitStiva(var S: stiva);
begin S.n:=0 end;
function EsteGoalaStiva(var S: stiva): Boolean;
{ folosim "var" pentru economie }
begin EsteGoalaStiva:=S.n=0 end;
procedure PuneInStiva(var S: stiva; elem: Integer);
begin
    with S do
        if n=max then WriteLn('Stiva plina !')
        else begin Inc(n); x[n]:=elem end
end;
procedure ScoateDinStiva(var S: stiva; var elem: Integer);
begin
    with S do
        if n=0 then WriteLn('Stiva goala !')
        else begin elem:=x[n]; Dec(n) end
end;
function Factorial(n: Integer): Integer;
var S: stiva; F: Integer;
begin
    InitStiva(S); F:=1;
    repeat
        PuneInStiva(S,n); Dec(n)
    until n=0;
    F:=1;
    repeat
        ScoateDinStiva(S,n); F:=F*n
    until EsteGoalaStiva(S);
    Factorial:=F
end;
var n: Integer;
begin
    WriteLn('Calcul factorial - simulare recursivitate');
    Write('Dati n = '); ReadLn(n);
    WriteLn(n, '!' = ', Factorial(n)); ReadLn
end.
```

Asupra stivelor vom reveni în lecția următoare, odată cu prezentarea tehnicii de programare "back-tracking", precum și în lecția 8, unde vor fi prezentate structurile dinamice de date.

Structurile de date folosite până acum sunt *statice*. Astfel, noi nu putem calcula, cu programul de mai înainte, factorialul unui număr mai mare decât constanta `max`.

Recursivitate directă și indirectă

- Toate subprogramele recursive prezentate până acum se autoapelau în mod *direct*, motiv pentru care se spune că s-a folosit **recursivitatea directă**.
 - Există, însă, și situații când un subprogram *P* va apela la el însuși din cadrul altui subprogram *Q*, apelat de *P*. Astfel, cele două subprograme se apelează reciproc. Conform regulilor de sintaxă și topică ale limbajului, cum *P* apelează pe *Q*, înseamnă că *Q* va trebui să apară înaintea lui *P*. Or și *Q* apelează pe *P*, iar pentru a soluționa acest impas, se declară unul din subprograme înaintea celuilalt: se scrie doar antetul său, urmat de cuvântul rezervat **forward**. Apoi, când se definește subprogramul declarat înainte ("forward"), în scrierea antetului se poate renunța la lista parametrilor.
- Spunem că avem de a face cu o **recursivitate indirectă** sau **încrucișată**.

Iată un exemplu:

```
program RecursivitateIndirecta;
function F(x: Integer): Integer; forward;
function G(x: Integer): Integer;
begin if x=0 then G:=1 else G:=G(x-1)+F(x-1) end;
function F;
begin if x=0 then F:=1 else F:=F(x-1)+G(x-1) end;
begin
    WriteLn(F(5));
    ReadLn
end.
```

Programul va afișa valoarea 32. Exemplul prezentat este pur teoretic, noi vom încerca să evităm utilizarea acestui gen de recursivitate, deoarece de multe ori duce la programe mai greu de urmărit.

Exerciții

Exercițiile următoare se cer a fi rezolvate folosind subprograme recursive:

1. Să se calculeze suma $S(n) = 1 + 3 + 5 + \dots + (2n-1)$.
2. Să se calculeze produsele $P_1(n) = 1 \times 4 \times 7 \times \dots \times (3n-2)$ și $P_2(n) = 2 \times 4 \times 6 \times \dots \times (2n)$.
3. Să se determine produsul componentelor unui vector.
4. Să se inverseze un șir de caractere.
5. Se consideră declarația de tip: `type vector=array[1..20] of Integer`. Să se verifice apartenența unui element la un vector `var x: vector`.
6. Să se verifice dacă un vector conține cel puțin un număr negativ în primele *n* poziții.
7. Să se afișeze conținutul unui vector.
8. Să se inverseze un vector.

5. Aplicație: evaluator de expresii algebrice

Descrierea algoritmului

În cele ce urmează ne propunem să realizăm un program care să citească expresia oricărei funcții și să o poată tabela, adică, dându-i-se un număr de puncte dintr-un interval, să afișeze valorile funcției în fiecare din acele puncte. Expresia funcției va fi memorată într-un șir de caractere. Ea va putea conține operații și funcții elementare (adunare, scădere, înmulțire, împărțire, ridicare la putere, `exp`, `ln`, `abs`, `radical`, `sin` și `cos`). Pentru a putea utiliza inclusiv funcții "cu acoladă", vom folosi o convenție din limbajul *BASIC*.

Astfel, să considerăm următoarele două funcții:

$$f(x) = \begin{cases} x \cdot \cos(1/x), & \text{dacă } x \neq 0 \\ 0, & \text{dacă } x = 0. \end{cases}$$

$$f(x) = \begin{cases} x^2, & \text{dacă } x \geq 0 \\ x - \sin(x \cdot 2), & \text{altfel} \end{cases}$$

Prima funcție va fi dată sub forma: $(x \neq 0) * (x * \cos(1/x)) + (x = 0) * 0$; iar a doua sub forma: $(x > 0) * (x^2) + (x < 0) * (x - \sin(x * 2))$;

Deci se vor folosi simbolurile "<" (mai mic sau egal), ">" (mai mare sau egal), "=" (egal) și "#" (diferit), precum și convenția din limbajul BASIC, anume că expresiile relaționale se evaluează la 1, dacă sunt adevărate, respectiv la 0, dacă sunt false.

Evaluarea funcției într-un punct x are la bază următoarea idee:

- de fiecare dată când în expresia funcției apare litera "x", aceasta va fi înlocuită cu valoarea lui x ;
- se vor folosi două stive: una a operanzilor (Opd - cu vârful $top1$) și una a operatorilor (Op - cu vârful $top2$);
- până la întâlnirea unei paranteze închise, care ar semnala închiderea unui bloc, cu stivele se lucrează în maniera următoare:
 - * de fiecare dată când este nevoie, se pune în stiva Opd operandul întâlnit;
 - * se pune în stiva Op , la început, '(', apoi, pe rând, operatorii întâlniți, doar dacă în vârful stivei nu este un operator de prioritate mai mare sau egală; în caz contrar, se scot operatorul din vârful stivei și cei doi operanzi din vârful stivei operanzilor, punându-se în stiva operanzilor rezultatul obținut prin aplicarea operatorului asupra celor doi operanzi;
- procedeul se repeta până când stiva operatorilor este vidă și nimic nu mai e de depus; în acest moment, stiva operanzilor va conține exact valoarea funcției în punctul respectiv.

Să exemplificăm funcționarea algoritmului anterior pentru expresia " $x + \text{abs}(x - 2 * 3)$ " și valoarea lui $x = 5$.

În primul rând, expresiei i se va adăuga încă o paranteză ")", după ce i-au fost înserate spații între *cuvinte*. Apoi, se desparte în cuvinte expresia, memorând cuvintele într-un vector. Prin "*cuvânt*" se va înțelege orice element distinctiv, adică numerele, variabila ("x"), operatorii și funcțiile, parantezele.

Astfel, pentru expresia din exemplul nostru, vom obține următorul vector de lungime 11:

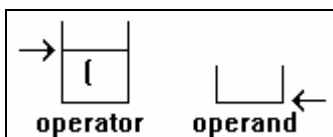
x	+	abs	(	x	-	2	*	3	)	)
---	---	-----	---	---	---	---	---	---	---	---

Lucrul cu stiva operatorilor depinde de un tabel de priorități care se asociază diferitelor operații:

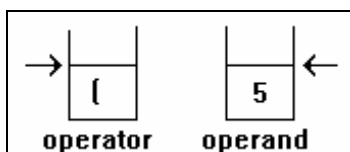
Tabelul priorităților

(	)	+	-	*	/	^	#	=	<	>	cos	sin	ln	exp	abs	rad
0	1	2	3	4				5								

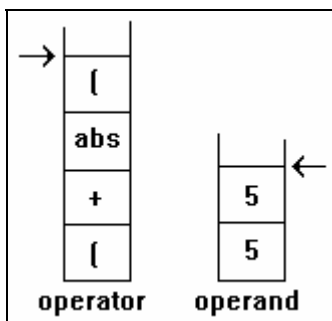
La început, stivele arată astfel:



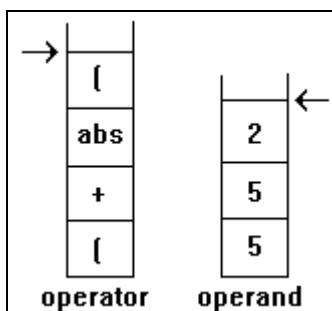
Prima componentă a vectorului nu este număr, ci este “x”. Deci se evaluează x, adică în stiva operanzilor se introduce valoarea sa, 5:



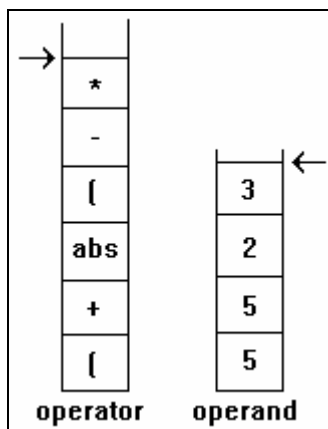
Apoi “+” se introduce în stiva operator, prioritatea lui “+” fiind 1, mai mare decât 0, care e prioritatea lui “(”, simbolul din vârful stivei operatorilor. Urmează “abs”, care are prioritatea 5, deci mai mare decât a lui “+”, așadar se introduce tot în stivă. Paranteza “(” se va pune și ea în stivă. Ea marchează începerea unui nou bloc. E rândul lui “x” să fie pus în stiva operanzilor, deci se va pune 5, iar stivele vor arăta acum astfel:



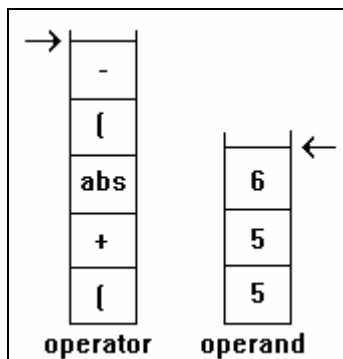
Urmează “-”, de prioritate $1 \geq \text{prioritate}('(')$, deci se pune și el în stiva operatorilor. În fine, se trece 2 în stiva operanzilor și, acum, când stivele arată astfel:



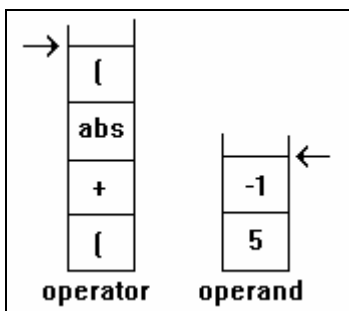
sosește operatorul “*”, prioritar față de “-”. Deci se va introduce în stiva operatorilor, iar numărul 3 în stiva operanzilor:



Urmează “)”, deci se aplică “*” asupra lui 3 și 2, punându-se rezultatul $3 \cdot 2 = 6$ în stiva operanzilor:

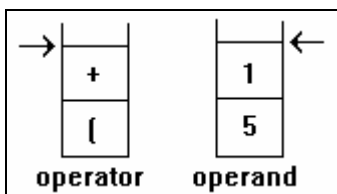


Paranteza “)” încheie blocul, deci se aplică “-” lui 5 și 6. Se obține:

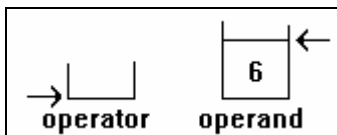


S-a terminat cu blocul “ $(x - 2 \cdot 3)$ ”, deci se elimină “(” și apoi aplicăm “abs” asupra vârfului stivei operanzilor, iar “abs” fiind operator unar, se va scoate un element din stiva operanzilor (-1) și se va introduce înapoi tot unul ($|-1| = 1$).

Stivele vor arăta astfel:



Se adună 1 cu 5, se scoate "+", apoi "(", iar stivele finale sunt:



Acum, stiva operanzilor conține valoarea funcției date, în punctul $x=5$, adică 6.

Protecția la erori

Programul care implementează algoritmul descris anterior folosește o serie de operații și funcții aritmetice elementare, care au fost rescrise pentru a proteja programul de eventualele erori de genul:

- adunarea, scăderea, înmulțirea, împărțirea a două numere foarte mari (motiv pentru care a fost definită o constantă `infinif`);
- extragerea rădăcinii pătrate dintr-un număr negativ
- calcularea logaritmului natural a unui număr negativ;

Aceste operații, împreună cu cele relaționale (implementate de funcțiile booleene `MaiMic`, `MaiMare` etc. și de `DifInf` (care verifică dacă un număr este diferit de infinit sau nu)) au fost scrise într-un fișier separat, numit `OPERATII.INC`. Acest fișier se include în textul programului principal prin directiva de compilare `{ $I nume_fis }`.

Iată fișierul `OPERATII.INC`:

```
{ OPERATII.INC }
const infinif: Real = 30000; epsi: Real=0.0001;
const OperatiiBinare: set of Char = ['+', '-', '*', '/', '^',
                                     '<', '>', '=', '#'];
      OperatiiUnare: set of Char = ['s', 'c', 'a', 'r', 'e', 'l'];
function DifInf(x: Real): Boolean;
begin
  DifInf := Abs(infinif-Abs(x)) > infinif / 2 end;
function Logarithm(x: Real): Real;
begin
  if (x>epsi) and DifInf(x) then Logarithm:=Ln(x)
  else Logarithm:=infinif {-}
end;
function Exponential(x: Real): Real;
begin
  if DifInf(x) then Exponential:=exp(x) else Exponential:=infinif
end;
function Inmultit(x,y: Real): Real;
begin
  if (Abs(x)<epsi) or (Abs(y)<epsi) then Inmultit:=0
  else if DifInf(x) and DifInf(y) then Inmultit:=x*y
  else Inmultit:=infinif
end;
function Putere(x,y: Real): Real;
var i,yi:integer; p:real;
```

```

begin
  if x=0 then Putere:=0
  else if y=0 then Putere:=1
  else
    if (x=infinif) or (y=infinif) then Putere:=Infinif
    else
      if y=Int(y) then
        begin
          yi:=Trunc(y); p:=1;
          for i:=1 to yi div 2 do p:=p*x; p:=p*p;
          if odd(yi) then p:=p*x; Putere:=p
          end
        else Putere:=Exponential(Inmultit(y,Logaritm(x)))
      end;
function Egal(x,y: Real): Real;
begin if x=y then egal:=1 else egal:=0 end;
function Diferit(x,y: Real): Real;
var df: Integer;
begin if x=y then df:=0 else df:=1; diferit:=df {1-egal(x,y)} end;
function MaiMic(x,y: Real): Real;
begin if x<y then maimic:=1 else maimic:=0 end;
function MaiMare(x,y: Real): Real;
begin if x>y then maimare:=1 else maimare:=0 end;
function Plus(x,y: Real): Real;
begin if DifInf(x) and DifInf(y) then Plus:=x+y else Plus:=infinif end;
function Minus(x,y: Real): Real;
begin if DifInf(x) and DifInf(y) then Minus:=x-y else Minus:=infinif
end;
function Impartit(x,y: Real): Real;
begin if abs(y)>epsi then Impartit:=x/y else Impartit:=infinif
end;
function Sinus(x: Real): Real;
begin if DifInf(x) then Sinus:=sin(x) else Sinus:=infinif end;
function Cosinus(x: Real): Real;
begin if DifInf(x) then Cosinus:=cos(x) else Cosinus:=infinif end;
function Modul(x: Real): Real;
begin if DifInf(x) then Modul:=abs(x) else Modul:=infinif end;
function Radical(x: Real): Real;
begin if DifInf(x) and (x>epsi) then Radical:=Sqrt(x) else
Radical:=infinif end;

```

Textul programului

Programul complet pentru evaluarea unei funcții într-un punct este dat mai jos. O expresie este memorată sub forma unei structuri de tip înregistrare:

```

{$X+}
program EvaluatorAlgebraic;
uses Crt;
const max=30;
type functie = record
  expresie: String;
  {expresia functiei = un sir de caractere}
  vector: array[1..max] of String[10];
  {vectorul cuprinzand "cuvintele"ce formeaza expresia}
  lung: Integer; {lungimea efectiva a vectorului}
  a,b: Real; {intervalul de evaluare a functiei}
  n: Integer {numarul de puncte de evaluare}
end;
{$I OPERATII.INC}
function Lower(s: String): String;
{ converteste in litere mici si transforma:
  - in 0- ; (- in (0- ; + in 0+ si (+ in (0+ }
var t: String; i: Byte;
begin
  t:=''; if s[1] in ['-','+','] then s:='0'+s;

```

```

i:=1;
while i < Length(s) do
  if (Copy(s,i,2)='-') or (Copy(s,i,2)='+') then
    begin Insert('0',s,i+1); Inc(i,2) end
  else Inc(i);
for i:=1 to Length(s) do
  if s[i] in ['A'..'Z'] then
    t:=t+Chr(Ord(s[i])+Ord('a')-Ord('A'))
  else t:=t+s[i];
Lower:=t
end;
function EsteIntreg(v:real):boolean;
begin
  EsteIntreg := (v=Int(v)) or (v=0)
end;

```

Pentru a despărți diferitele entități ale unei expresii, sunt necesare să se identifice “tipurile” a câte două caractere vecine din șirul reprezentând expresia algebrică. Dacă t_1 și t_2 sunt tipurile a două caractere vecine din expresie, atunci, dacă ele sunt diferite, înseamnă că nu fac parte din aceeași entitate, prin urmare vor fi cuprinse în două “cuvinte” diferite, motiv pentru care se va insera un spațiu între ele.

```

function TipCaracter(c: Char): Byte;
var tip: Byte;
begin
  case c of
    '0'..'9','.': tip:=1; { cifre }
    'c','o','s','i','n','l','e','x','p',
    'a','b','r','d': tip:=2;
    { cos, sin, ln, exp, abs, rad }
    '(': tip:=3; { paranteze }
    ')': tip:=4;
    '+': tip:=5; { operatii aritmetice }
    '-': tip:=6;
    '*': tip:=7;
    '/': tip:=8;
    '^': tip:=9;
    'x': tip:=10; { variabila }
    'q': tip:=11; { pi }
    '=': tip:=12;
    '#': tip:=13;
    '<': tip:=14;
    '>': tip:=15
  else tip:=100
  end;
  TipCaracter:=tip
end;
function EsteNumar(sir: String): Boolean;
var valoare: Real; cod: Integer;
begin
  Val(sir,valoare,cod); EsteNumar := cod=0
end;

```

Procedura următoare inserează spații în expresia funcției, între caracterele de tipuri diferite. Din $\sin(x+3.2)$ se va obține $\sin (x + 3.2)$.

```

procedure PuneSpatiiInExpresie(var E: functie);
{ insereaza spatii in expresia functiei,
  intre caractere de tipuri diferite }
var i,t1,t2: Byte;
begin
  with E do
    begin { se pune X in loc de x, dar nu si la functia exp ! }
      if expresie[1]='x' then expresie[1]:='X';
      if expresie[Length(expresie)]='x' then

```



```

        expresie[Length(expresie)]:='X';
    for i:=1 to Length(expresie) do
        if expresie[i]='x' then
            if (i<Length(expresie)) and (i>1) then
                if (expresie[i-1]<>'e') and (expresie[i+1]<>'p') then
                    expresie[i]:='X';
            end;
        end;
    end;
    i:=1;
    while (i<Length(expresie)) and (Length(expresie)<256) do
        begin
            t1:=TipCaracter(expresie[i]);t2:=TipCaracter(expresie[i+1]);
            if not ( (t1 in [5,6]) and (t2=1) and
                (expresie[i-2] in ['=','#','<','>']) ) then
                if (t1<>t2) or ((t1=TipCaracter('(')) and (t2=t1)) or
                    ((t1=TipCaracter(')')) and (t2=t1)) then
                    begin Insert(' ',expresie,i+1); i:=i+1 end;
                end;
            i:=i+1
        end
    end;
end;

```

Despărțirea în cuvinte se realizează cu procedura următoare:

```

procedure TransformaInVector(var E: functie);
{ desparte in cuvinte pe E.expresie, memorand cuvintele in
  E.vector[1..lung] }
var p: Byte; sir: String;
begin
    with E do
        begin
            expresie := expresie+' '; sir:=expresie; lung:=0;
            while sir<>' ' do
                begin
                    p := Pos(' ',sir); lung := lung+1;
                    vector[lung] := Copy(sir,1,p-1); Delete(sir,1,p)
                end
            end;
        end
    end;
end;

```

Tabelul priorităților se regăsește în funcția de mai jos:

```

function Prioritate(c: Char): Byte;
var p: Byte;
begin
    case c of
        '(', ')': p:=0;
        '+', '-': p:=1;
        '*', '/': p:=2;
        '^': p:=3; { ridicarea la putere }
        '=', '#', '<', '>': p:=4; { operatori relationali }
        'c', 's', 'l', 'e', 'a', 'r': p:=5 { cos, sin, ln, exp, abs, rad }
    else p:=5
    end;
    Prioritate := p
end;

```

Evaluarea propriu-zisă a funcției:

```

function ValoareFunctie(E: functie; x: Real): Real;
{ returneaza valoarea functiei E in punctul x }
const max_stiva=100;
var valo,x_1,x_2: Real; i,cod,top1,top2:integer;
    { varfurile celor doua stive }
    Opd: array [1..max_stiva] of Real; { stiva operanzilor }
    Op: array [1..max_stiva] of Char; { stiva operatorilor }
begin { calculul expresiei }
    for i:=1 to max_stiva do
        begin Opd[i]:=0; Op[i]:='@' end;
    top1:=0; top2:=1; Op[top2]:='('; i:=0;

```

```

with E do
  while (i<=lung) and (top2>0) do
    begin
      Inc(i);
      if EsteNumar(vector[i]) then
        begin
          Inc(top1); Val(vector[i],valo,cod);
          Opd[top1]:=valo
        end
      else
        case vector[i][1] of
          { constanta pi=3.1415926 se da sub forma literei q }
          'q': begin Inc(top1); Opd[top1]:=pi end;
          'X': begin { variabila x } Inc(top1); Opd[top1]:=x end;
          '(': begin {inceput de bloc} Inc(top2); Op[top2]:='(' end
          else
            begin { operatii binare si unare }
              while (top2>0) and (not (Op[top2] in ['(',')']))
                and (Prioritate(Op[top2]) >=
                  Prioritate(vector[i][1])) do
                begin
                  if top1>1 then x_1:=Opd[top1-1];
                    x_2:=Opd[top1];
                    { functii scrise in OPERATII.PAS }
                    case Op[top2] of
                      '=': valo:=Egal(x_1,x_2);
                      '#': valo:=Diferit(x_1,x_2);
                      '<': valo:=MaiMic(x_1,x_2);
                      '>': valo:=MaiMare(x_1,x_2);
                      '+': valo:=Plus(x_1,x_2);
                      '-': valo:=Minus(x_1,x_2);
                      '*': valo:=Inmultit(x_1,x_2);
                      '/': valo:=Impartit(x_1,x_2);
                      '^': valo:=Putere(x_1,x_2);
                      's': valo:=Sinus(x_2);
                      'c': valo:=Cosinus(x_2);
                      'l': valo:=Logaritm(x_2);
                      'e': valo:=Exponential(x_2);
                      'a': valo:=Modul(x_2);
                      'r': valo:=Radical(x_2);
                    end; { case operator[top2] }
                    if Op[top2] in OperatiiBinare then
                      Dec(top1);
                    if Op[top2] in
                      (OperatiiUnare+OperatiiBinare) then
                      Opd[top1]:=valo;
                      Dec(top2);
                    end; { while }
                    if top2>0 then
                      if (Op[top2]<>'(') or (vector[i]<>')') then
                        begin Inc(top2); Op[top2] := vector[i][1] end
                      else Dec(top2);
                    end; { else - operatii }
                    end { case vector[i][1] }
                end; { while }
                if top2=0 then ValoareFunctie := Opd[1]
                else ValoareFunctie := infinit
            end; {calculul_expresiei}

```

Procedura următoare citește date despre funcție: expresia sa, intervalul de tabelare, precum și numărul de puncte în care se vor calcula valorile funcției, din intervalul precizat.

```

procedure CitesteFuncție(var E: funcție);
begin
  with E do
    begin
      WriteLn('Dati expresia !'); Write('f(x)=');
      ReadLn(expresie);
      expresie:=Lower(expresie); WriteLn('Dati intervalul !');
      ReadLn(a,b); Write('Dati nr. de puncte = '); ReadLn(n)
    end;
    PuneSpatiiInExpresie(E); TransformaInVector(E)
  end;
end;

```

Următoarea procedură tablează o funcție E:

```

procedure AfiseazaValoriFuncție(var E: funcție);
{ E este transmis prin referință, pentru economie de memorie }
var i: Integer; x,y: Real;
begin
  for i:=0 to E.n-1 do
    begin
      x:=E.a+i*(E.b-E.a)/(E.n-1); y:=ValoareFuncție(E,x);
      if DifInf(y) then WriteLn(i+1:3,'. ',x:10:7,' -> ',y:10:7)
      else WriteLn(i+1:3,'. ',x:10:7,' -> nedefinit !');
      if (i+1) mod 24 = 0 then ReadKey
    end;
    ReadKey
  end;
  var E: funcție;
  begin
    CitesteFuncție(E);
    AfiseazaValoriFuncție(E)
  end.
end.

```



6. Exerciții recapitulative

1. Fie funcția :

```

function Invers(S: String): String;
var t: String; i: Integer;
begin
  t:='';
  for i:= Length(s) downto 1 do t:=t+s[i];
  Invers:=t
end;

```

Scrieți o funcție: `function EstePalindrom(S: String): Boolean;`
 astfel încât `EstePalindrom('AERISIREA') = True`,
 iar `EstePalindrom('ABRACADABRA') = False`.

2. Rescrieți subprogramele de lucru cu șiruri de caractere:

3. Să se definească un tip de date pentru numere naturale foarte mari și să se scrie proceduri care să adune și să scadă astfel de numere.

4. Elaborați o funcție care să determine diferența între două momente de timp date prin oră, minute și secunde.

5. Scrieți un program care să tableze o funcție, definită în textul programului, adică să afișeze valorile funcției în n puncte dintr-un interval dat $[a, b]$.

6. Îmbunătățiți programul `AdmitereLiceu` (din secțiunea 3 a acestei lecții) pentru a avea posibilitatea de a șterge, insera un elev sau modifica datele despre un elev. De asemenea, programul să permită listarea tuturor elevilor care au mediile cuprinse între două valori date.

7. Este consistentă următoarea definiție recursivă:

```

function Test(x: Integer);
begin
  if x=0 then Test:=1
  else if x=1 then Test:=Test(x+1) else Test:=Test(x+2)+x
end;

```

8. Ce afișează programul de mai jos:

```

program Test;
var x: Integer;
function F(x: Integer): Integer;
begin x:=x+1; F:=x*x end;
begin x:=2; WriteLn(x,':',F(x),':',x) end.

```

Dar dacă F ar avea antetul `function F(var x: Integer): Integer` ?

9. Ce erori sintactice sunt în programul de mai jos ?

```

program Test;
function F(var x: Integer): Integer;
begin x:=x+1; F:=x end;
begin
  WriteLn(2,':',F(2))
end.

```

10. Rezolvă problema mutării a n discuri de pe turnul p pe turnul q , în problema Turnurilor din Hanoi, următoarea procedură? Este ea o procedură definită consistent ?

```

procedure Hanoi2(n,p,q: Integer);
var r: Integer;
begin
  r:=6-p-q;
  if n=1 then WriteLn(p,'-',q)
  else
    begin
      WriteLn(p,'-',r); Hanoi2(n-1,p,r); WriteLn(r,'-',q)
    end
  end.

```

Semnificația procedurii este: pentru a muta n discuri de la turnul p la turnul q se mută discul de deasupra de pe turnul p pe turnul r , apoi se mută, recursiv, cele $n-1$ discuri de pe turnul p pe turnul auxiliar r , după care se aduce discul de pe q pe r .

11. Scrieți o procedură iterativă care să calculeze, cu ajutorul unei stive, suma $S=1+2+3+\dots+n$, respectiv produsul $P=1\times3\times5\times\dots\times(2n-1)$.

12. Funcționează programul următor? Dacă da, ce va afișa?

```

program Test;
var y,x: Integer;
function F(var z,x: Integer): Integer;
begin z:=x-y; x:=x+1; F:=x+y+z end;
begin y:=2; x:=y+5; WriteLn(x,':',y,':',F(x,y),':',x,':',y); end.

```

13. Scrieți un program care să afișeze literele unei propoziții la întâmplare pe ecran, apoi să le deplaseze către poziția lor finală, pentru ca șirul să fie scris pe linia din mijloc a ecranului.

14. Ce va afișa programul următor? Ar fi posibil un antet cu `var` pentru funcția G ?

```

program Test;
function F(var x: Integer): Integer;
begin x:=x+1; F:=x end;
function G(x,y: Integer): Integer;
begin G:=x+y end;
var y: Integer;
begin y:=2; WriteLn(G(F(y),F(y))) end.

```

- 15.** Un triunghi este definit prin coordonatele vârfurilor sale. Scrieți funcții care, pentru două triunghiuri date, să studieze dacă: a) au aceeași arie; b) sunt asemenea; c) primul este în interiorul celui de al doilea.
- 16.** Fie o tablă de șah de dimensiune $n \times n$, pe care sunt plasate două dame, prima pe linia `l1` și coloana `c1`, iar a doua pe linia `l2` și coloana `c2`. Scrieți o funcție booleană care să verifice dacă damele se atacă între ele sau nu.
- 17.** Scrieți o procedură `Divide(a,b: Integer; c,d: Integer)` care să împartă, prin scăderi repetate, pe `a` la `b`, obținându-se câtul `c` și restul `d`.
- 18.** Scrieți o procedură care pune într-un vector, în primele `n` componente, primele `n` numere prime care, citite invers, sunt tot numere prime.